

Java OOP Practice Problems

Class, Constructor, Method, Inheritance, Abstraction, Polymorphism and More

Problems only. No answers included.

Problem List

No.	Topic	Main Concept
1	Class and Object	Class and Object
2	Constructor	Constructor
3	Method	Method
4	Constructor Overloading	Constructor Overloading
5	Encapsulation	Encapsulation
6	Inheritance	Inheritance
7	Multilevel Inheritance	Multilevel Inheritance
8	Method Overriding	Method Overriding
9	Method Overloading	Method Overloading
10	Abstraction	Abstraction
11	Interface	Interface
12	Polymorphism	Polymorphism
13	Super Keyword	Super Keyword
14	Final Keyword	Final Keyword
15	Static Variable and Static Method	Static Variable and Static Method
16	Composition	Composition
17	Array of Objects	Array of Objects
18	Banking System Mini Problem	Banking System Mini Problem
19	University Management Mini Problem	University Management Mini Problem
20	Online Shopping Mini Problem	Online Shopping Mini Problem
21	Library Management Mini Problem	Library Management Mini Problem
22	Abstract Class and Interface Together	Abstract Class and Interface Together
23	Polymorphism with Array	Polymorphism with Array
24	Exception with OOP	Exception with OOP
25	Complete OOP Challenge	Complete OOP Challenge

Practice Problems

1. Class and Object

- Create a class named Student with the attributes name, id, department, and cgpa.
- Create objects of the Student class.
- Write a method called displayInfo() to display the student details.

2. Constructor

- Create a class named Book with the attributes title, author, and price.
- Use a constructor to initialize the values.
- Create three Book objects and print their details.

3. Method

- Create a class named Calculator.
- Add methods for addition, subtraction, multiplication, and division.
- Take two numbers as input from the user and call the methods to show the results.

4. Constructor Overloading

- Create a class named Employee with the attributes name, id, and salary.
- Create one default constructor.
- Create one parameterized constructor with all attributes.
- Create one constructor with only name and id.
- Display employee details using a method.

5. Encapsulation

- Create a class named BankAccount.
- Use private attributes: accountNumber, accountHolderName, and balance.
- Use getter and setter methods.
- Add a deposit() method and a withdraw() method.

6. Inheritance

- Create a parent class named Vehicle with the attributes brand, model, and year.
- Create a child class named Car that inherits from Vehicle.
- Add an extra attribute called numberOfDoors in the Car class.
- Display all details of the car.

7. Multilevel Inheritance

- Create a class named Person.
- Create a child class named Student that inherits from Person.
- Create another child class named GraduateStudent that inherits from Student.
- Add suitable attributes and methods for each class.

8. Method Overriding

- Create a parent class named Animal with a method called sound().
- Create child classes named Dog, Cat, and Cow.
- Override the sound() method in each child class.

9. Method Overloading

- Create a class named AreaCalculator.
- Create overloaded methods called calculateArea().
- Calculate the area of a square, rectangle, and circle using the overloaded methods.

10. Abstraction

- Create an abstract class named Shape.

- Add an abstract method called `calculateArea()`.
- Create child classes named `Circle`, `Rectangle`, and `Triangle`.
- Each child class should implement the `calculateArea()` method.

11. Interface

- Create an interface named `Payment`.
- Add a method called `makePayment()`.
- Create classes named `BkashPayment`, `NagadPayment`, and `CardPayment`.
- Each class should implement the `makePayment()` method.

12. Polymorphism

- Create a parent class named `Employee`.
- Add a method called `calculateSalary()`.
- Create child classes named `FullTimeEmployee`, `PartTimeEmployee`, and `ContractEmployee`.
- Each child class should calculate salary differently.

13. Super Keyword

- Create a parent class named `Teacher` with the attributes `name` and `subject`.
- Create a child class named `MathTeacher`.
- Use the `super` keyword to call the parent class constructor.

14. Final Keyword

- Create a class named `University`.
- Add a final variable called `universityName`.
- Try to change the value of the final variable and observe the result.
- Create a final method and try to override it in a child class.

15. Static Variable and Static Method

- Create a class named `Counter`.
- Use a static variable to count how many objects are created from the class.
- Create a static method to display the count.

16. Composition

- Create a class named `Engine` with the attributes `engineType` and `horsePower`.
- Create another class named `Car`.
- The `Car` class should have an object of `Engine`.
- Display car and engine details.

17. Array of Objects

- Create a class named `Product` with the attributes `productId`, `productName`, and `price`.
- Create an array of 5 `Product` objects.
- Display all product details.

18. Banking System Mini Problem

- Create a simple banking system.
- Use the classes `Account`, `SavingsAccount`, and `CurrentAccount`.
- Use inheritance.
- Add methods for `deposit`, `withdraw`, and `check balance`.
- Savings accounts and current accounts should have different withdrawal rules.

19. University Management Mini Problem

- Create a university management system.
- Use the classes `Person`, `Student`, `Teacher`, and `Course`.
- Use inheritance and object relationships.

- Students should be able to enroll in courses.
- Teachers should be assigned to courses.

20. Online Shopping Mini Problem

- Create an online shopping system.
- Use the classes Product, Customer, Cart, and Order.
- The customer should be able to add products to the cart, remove products, and place an order.

21. Library Management Mini Problem

- Create a library management system.
- Use the classes Book, Member, and Library.
- Add methods for adding books, borrowing books, returning books, and displaying available books.

22. Abstract Class and Interface Together

- Create an abstract class named User with the attributes name and email.
- Create an interface named Loginable with a method called login().
- Create classes named Admin and Customer.
- Both classes should extend User and implement Loginable.

23. Polymorphism with Array

- Create a parent class named Shape.
- Create child classes named Circle, Rectangle, and Square.
- Create an array of Shape objects.
- Call the calculateArea() method for each object.

24. Exception with OOP

- Create a class named BankAccount.
- If the user tries to withdraw more money than the balance, throw a custom exception called InsufficientBalanceException.

25. Complete OOP Challenge

- Create a small school management system.
- Use the classes Person, Student, Teacher, Course, and Department.
- Use class and object, constructor, method, inheritance, method overriding, encapsulation, abstraction, and polymorphism.
- The system should display student, teacher, course, and department information.